

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

ÉTAPE 1 — FONDATION

Socle de compte unifié

Types de compte, vérification et badges

*Fondation commune aux comptes business, à la certification
et au compte officiel*



Périmètre Talky (Flutter) ▪ Alanya-Backend ▪ Alanya Admin

Statut Conception — aucune ligne de code écrite

Précède Listes de contacts, compte officiel, comptes business, certification

Date 26 juillet 2026

Document confidentiel — diffusion restreinte

Avant-propos

Quatre fonctionnalités ont été demandées pour Alanya : les comptes business, la certification des comptes, un compte officiel au nom de l'application, et les listes de contacts préférés.

Trois d'entre elles — business, certification, compte officiel — ne sont pas trois systèmes différents. Ce sont trois réponses à la même question : *qui est ce compte ?* Les concevoir séparément produirait trois systèmes de badges concurrents, trois écrans de profil et une logique d'affichage impossible à tenir. Ce document pose donc d'abord la fondation commune, et rien d'autre.

Comment lire ce document. Il est organisé en deux parties. La **partie I** décrit la fonctionnalité telle que l'utilisateur et le client la voient, sans vocabulaire technique. La **partie II** décrit comment on la construit. Les deux couvrent le même périmètre ; on peut valider la partie I sans lire la partie II.

Méthode. Toutes les affirmations sur l'existant de ce document ont été vérifiées en lisant le code des trois dépôts. Les chemins de fichiers cités sont réels. Là où une information manque, c'est écrit.

Table des matières

Avant-propos	1
I Approche naïve	4
1 Ce que l'on ajoute, et pourquoi	5
1.1 La situation aujourd'hui	5
1.2 Deux questions, pas une	5
1.3 Ce que couvre l'étape 1 — et ce qu'elle ne couvre pas	5
2 Ce que l'utilisateur voit	6
2.1 Le principe du badge	6
2.2 Les cinq rendus	6
2.3 Où le badge apparaît	7
2.4 Pourquoi la distinction déclaré / vérifié est critique	7
2.5 Protéger le badge des contournements	8
3 Les règles de vie d'un compte	9
3.1 Devenir un compte business, et revenir en arrière	9
3.2 La vie d'une vérification	9
3.3 L'expiration et sa relance	9
3.4 Le compte officiel	10
3.5 Un compte business ne peut pas être administrateur	10
II Approche technique	11
4 Modèle de données	12
4.1 Pourquoi étendre <code>users</code> plutôt que créer une table <code>accounts</code>	12
4.2 La migration	12
4.3 Les énumérations, côté Node	13
4.4 Raccordement au schéma existant	13
5 Dérivation du badge	14
5.1 Une fonction pure, trois implémentations identiques	14
5.2 Le piège à corriger avant tout	15
5.3 Composant existant à ne pas confondre	15

6	Règles appliquées par le serveur	16
6.1	Exclusion business / administrateur	16
6.2	Validation des noms d’affichage	16
6.3	Interdire la réponse au compte officiel	17
6.4	Expiration : réutiliser le mécanisme existant	17
7	Impact fichier par fichier	19
7.1	Backend — Alanya-Backend	19
7.2	Mobile — Talky (Flutter)	20
7.3	Administration — Alanya Admin	20
8	Points de vigilance et questions ouvertes	21
8.1	À vérifier avant d’appliquer la migration	21
8.2	Collision de numérotation	21
8.3	Écart assumé avec la conception initiale	21
8.4	Ce qui reste à décider	21

PARTIE I

Approche naïve

CHAPITRE 1

Ce que l'on ajoute, et pourquoi

1.1 La situation aujourd'hui

Dans Alanya, tous les comptes sont identiques. Un commerçant, une personnalité publique et un utilisateur ordinaire ont exactement le même profil et la même apparence dans une liste de conversations. Rien ne permet de distinguer l'entreprise officielle d'un imitateur qui aurait pris le même nom et la même photo.

C'est ce manque que les trois fonctionnalités demandées viennent combler — et c'est pourquoi elles partagent la même fondation.

1.2 Deux questions, pas une

L'erreur naturelle serait de créer une liste de « genres de comptes » : personnel, business, vérifié, officiel, business vérifié... Cette liste devient ingérable dès qu'on ajoute un cas.

On sépare donc deux questions **indépendantes** :

Quel genre de compte ?	Personnel, business, ou officiel. C'est ce que le compte <i>est</i> .
A-t-il été vérifié ?	Aucune démarche, en attente, vérifié, refusé, révoqué, ou expiré. C'est ce qu'Alanya <i>atteste</i> à son sujet.

Ces deux questions ne dépendent pas l'une de l'autre. Un commerce peut exister sans être vérifié. Une personne privée peut être vérifiée sans être un commerce — c'est le cas d'une personnalité publique. Le compte officiel Alanya, lui, est la combinaison « officiel + vérifié d'office ».

Le bénéfice concret. Poser deux questions séparées plutôt qu'une seule liste, c'est ce qui permet d'ajouter demain un nouveau genre de compte, ou un nouvel état de vérification, sans toucher à l'autre axe ni refaire les écrans.

1.3 Ce que couvre l'étape 1 — et ce qu'elle ne couvre pas

Dans cette étape	Savoir <i>qui</i> est un compte : son genre, son état de vérification, et le badge qui en découle partout dans l'application.
Étapes suivantes	Ce que ce compte <i>contient</i> : la fiche business (horaires, adresse, catalogue, liens), le dossier de certification (pièces justificatives, approbation, abonnement), et la diffusion du compte officiel.

La raison de cette coupure est simple : le genre et l'état d'un compte suffisent déjà à afficher le bon badge partout. Le contenu détaillé d'une fiche business dépend de règles qu'on n'a pas encore écrites — le figer maintenant obligerait à tout refaire à l'étape suivante.

CHAPITRE 2

Ce que l'utilisateur voit

2.1 Le principe du badge

Un badge qui combinerait « c'est un commerce » et « c'est vérifié » en un seul dessin devient illisible à la taille où il apparaît réellement — environ 12 pixels, à côté d'un nom dans une liste de conversations. Et deux badges côte à côte mangent la largeur du nom, qui est déjà tronqué sur un petit écran.

La solution retenue sépare les deux informations sur deux canaux visuels distincts :

Le glyphe dit <i>quoi</i>	Une coche pour une personne notable, un panier pour un commerce, la marque Alanya pour le compte officiel.
Le contenant dit <i>vérifié</i> ou <i>pas</i>	Un glyphe nu = déclaré par le compte lui-même. Un glyphe posé dans un sceau festonné  = attesté par Alanya.

Pourquoi un sceau festonné, et pas un rond

Un disque plein ne se distingue d'aucune autre icône circulaire de l'interface : avatars, puces de comptage, indicateurs de présence. À 12 pixels, un badge rond devient un point coloré parmi d'autres points colorés.

Le feston à douze lobes, lui, a une silhouette reconnaissable même quand le glyphe qu'il contient n'est plus lisible. C'est la propriété qui compte : l'utilisateur qui distingue la *forme* sait qu'Alanya s'est prononcé sur ce compte, sans avoir à lire le détail. Et cette forme est bien plus difficile à imiter avec un caractère Unicode dans un nom d'affichage qu'un simple rond ou une simple coche.

2.2 Les cinq rendus

Genre	État	À taille réelle	Agrandi
Personnel	non vérifié	—	—
Personnel	vérifié		
Business	déclaré		
Business	vérifié		
Officiel	—		

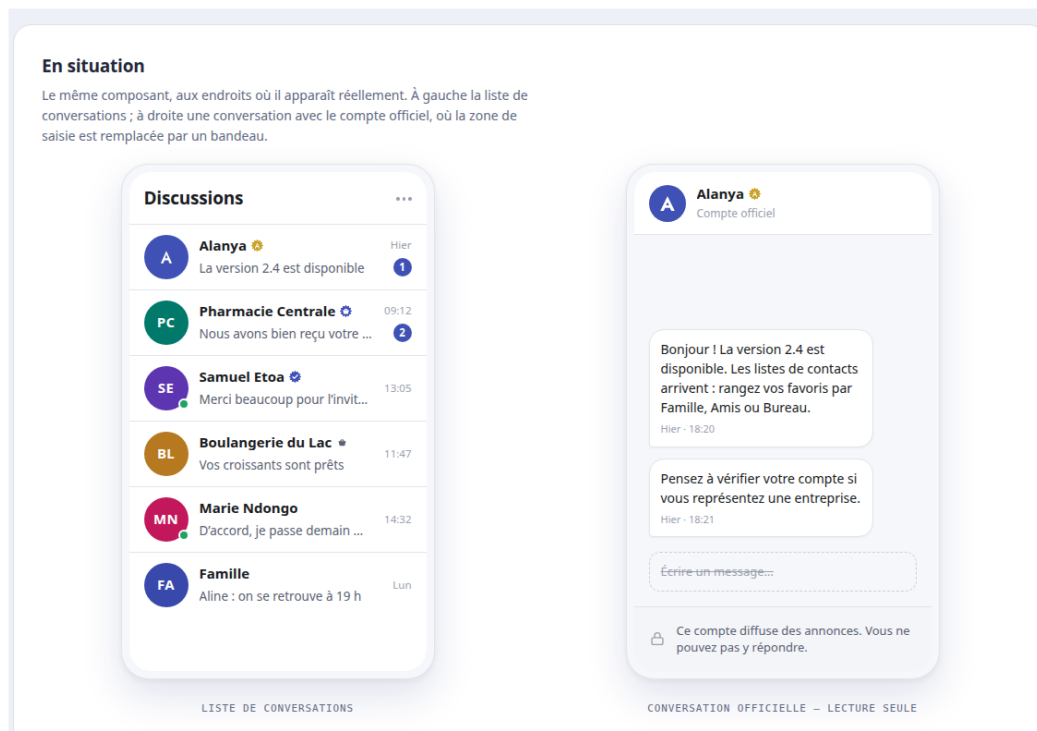
La colonne « à taille réelle » est là pour être jugée telle quelle : c'est exactement la taille à laquelle le badge apparaîtra à côté d'un nom. Si un rendu n'est pas distinguable à cette échelle, il doit être redessiné avant l'implémentation.

Pourquoi une couleur différente pour l'officiel. Le sceau du compte officiel est doré, pas indigo. Il n'existe qu'un seul compte officiel dans toute l'application : lui donner une couleur à part évite qu'il soit confondu avec un compte simplement vérifié, et rend l'usurpation visuellement plus difficile.

2.3 Où le badge apparaît

Le même badge, dessiné par le même composant, dans quatre contextes :

- la **liste de conversations**, juste après le nom ;
- l'**en-tête d'une conversation** ouverte ;
- la **page de profil** du compte ;
- les **résultats de recherche** et les listes de contacts.



Maquette interactive. La version vivante de cette maquette permet de faire varier la taille du badge de 10 à 28 pixels pour juger la lisibilité, et montre la conversation officielle avec son bandeau de lecture seule :

claude.ai/code/artifact/029627c6

Source versionnée : docs/architecture/maquettes/liste-conversations-badges.html

2.4 Pourquoi la distinction déclaré / vérifié est critique

Se déclarer « commerce » prend trente secondes et n'engage personne : le compte l'affirme lui-même, personne ne l'a contrôlé. Être vérifié suppose au contraire un dossier examiné par Alanya.

Si les deux badges avaient le même poids visuel, n'importe qui créerait un compte business et hériterait d'une caution qui ne lui a jamais été accordée. C'est exactement le scénario qu'un arnaqueur cherche.

La règle qui en découle. Le sceau festonné est la signature d'Alanya. Rien d'autre dans l'interface ne doit pouvoir lui ressembler — ni une icône décorative, ni un emoji dans un nom d'utilisateur.

2.5 Protéger le badge des contournements

Un système de badge peut être contourné sans jamais être attaqué techniquement : il suffit qu'un utilisateur s'appelle « Alanya ✓ » pour que le nom affiché imite la certification.

Deux protections sont donc posées dès cette étape :

1. les caractères de type coche, étoile et sceau sont **refusés** dans le nom et le pseudo ;
2. les variantes du nom « Alanya » sont **réservées** et ne peuvent pas être prises par un compte ordinaire.

CHAPITRE 3

Les règles de vie d'un compte

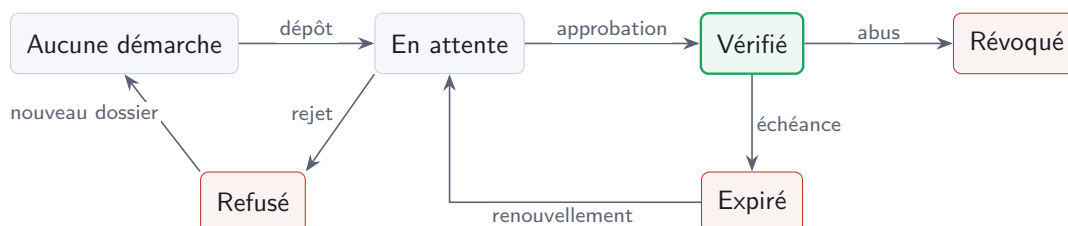
3.1 Devenir un compte business, et revenir en arrière

Le passage d'un compte personnel à un compte business est **réversible**. Les règles proposées :

Personnel → business	Libre, à l'initiative de l'utilisateur. Devenir commerce est déclaratif : cela n'atteste de rien et ne demande donc aucune validation.
Business → personnel	Libre également. La fiche business est conservée mais masquée , pas supprimée : un retour en arrière ne doit pas détruire un catalogue de produits saisi pendant des semaines.
Une vérification en cours	La demande est annulée : elle portait sur une identité d'entreprise qui n'est plus revendiquée.
Une vérification obtenue	Elle est révoquée , pas suspendue. Le badge atteste d'une entreprise ; si le compte n'en est plus une, l'attestation n'a plus d'objet. Un nouveau dossier sera nécessaire pour la retrouver.

À confirmer avec le client. La bascule business → personnel pourrait aussi être soumise à validation d'un administrateur, pour éviter qu'un commerce échappe à un signalement en repassant en personnel. La règle proposée ci-dessus est la plus simple ; l'alternative est plus sûre.

3.2 La vie d'une vérification



3.3 L'expiration et sa relance

Une vérification n'est pas éternelle : elle a une date de fin. Le comportement attendu :

À J – 30	Le titulaire est prévenu qu'il doit renouveler, par trois canaux à la fois : une notification sur son téléphone, un e-mail, et un avertissement visible dans l'application. L'avertissement n'est envoyé qu'une fois.
À l'échéance	La vérification est suspendue . Le badge disparaît. Le compte et son contenu restent intacts — seule l'attestation tombe.
Après	Le titulaire peut déposer un nouveau dossier pour retrouver son badge.

Suspendu, pas supprimé. Une vérification expirée est un état distinct d'un refus. Le compte a bien été vérifié par le passé ; il a simplement laissé passer la date. Cette nuance compte pour le service client, et pour ne pas retraiter un dossier déjà instruit.

3.4 Le compte officiel

Le compte officiel est le compte au nom de l'application. Ses règles :

- il est **créé par un administrateur**, jamais par une inscription ordinaire ;
- sa photo de profil est le **logo Alanya** ;
- il porte le badge officiel, distinct du badge de vérification ;
- les utilisateurs **ne peuvent pas lui répondre**. Ses messages sont des diffusions, pas des conversations.

Concrètement, dans une conversation avec le compte officiel, la zone de saisie est remplacée par un bandeau explicatif, et les actions « répondre » et « réagir » n'apparaissent pas sur ses messages.

Ce n'est pas qu'une question d'affichage. Masquer la zone de saisie ne suffit pas : un client modifié pourrait envoyer le message quand même. Le refus doit être appliqué par le serveur. La partie II décrit où.

3.5 Un compte business ne peut pas être administrateur

Décision prise : les rôles d'administration de la plateforme et les genres de compte sont mutuellement exclusifs. Un compte business ou officiel ne peut pas recevoir de droits d'administration, et réciproquement.

La raison est de sécurité : un compte visible du public, susceptible d'être usurpé ou compromis, ne doit pas ouvrir la porte au back-office.

PARTIE II

Approche technique

CHAPITRE 4

Modèle de données

4.1 Pourquoi étendre `users` plutôt que créer une table `accounts`

La conception initiale prévoyait une nouvelle table `accounts` servant de socle. La lecture du schéma réel invalide cette approche.

Constat vérifié dans le code. 14 tables référencent `users(alanyaID)` par clé étrangère : `preferredContact`, `blocked`, `userAccess`, `conversation`, `conv_participants`, `message`, `statut`, `statut_views`, `meeting`, `participant`, `callHistory`, `reserved_alanya_phone`, `user_push_devices` et `user_notification_prefs` — soit 34 contraintes au total (sources : `migrations/001_initial_schema.sql`, 010, 020, 021).

Introduire une table parallèle imposerait de réécrire ces 34 contraintes. On étend donc `users`, qui est le socle de compte.

Ce choix a un second bénéfice, qui n'est pas un hasard : la conception exige que le genre et l'état de vérification soient lisibles dans la requête de liste de conversations **sans jointure supplémentaire**. Cette requête joint déjà `users` pour récupérer le nom et l'avatar du correspondant (`src/controllers/conversationController.js`, ligne 24). Deux colonnes de plus sur `users` sont donc gratuites ; une table `accounts` aurait imposé une jointure de plus sur le chemin le plus chaud de l'application.

4.2 La migration

```
1  -- 023_account_socle.sql -- socle de compte unifie
2  --
3  -- Deux axes orthogonaux portés par 'users' :
4  --   account_type           : ce que le compte EST
5  --   verification_status    : ce qu'Alanya ATTESTE a son sujet
6  --
7  -- TINYINT et non ENUM : ajouter une valeur a un ENUM impose un ALTER
   TABLE
8  -- sur une table qui grossit. L'enumeration vit cote Node.
9
10 ALTER TABLE users
11   ADD COLUMN account_type TINYINT NOT NULL DEFAULT 0
12   COMMENT '0=personnel 1=business 2=officiel',
13   ADD COLUMN verification_status TINYINT NOT NULL DEFAULT 0
14   COMMENT '0=aucun 1=en_attente 2=verifie 3=refuse 4=revoque
15           5=expire',
16   ADD COLUMN verified_until DATETIME NULL
17   COMMENT 'Echeance de la verification (NULL si non verifie)',
18   ADD COLUMN verification_reminder_sent TINYINT NOT NULL DEFAULT 0
19   COMMENT '1 = relance J-30 deja envoyee pour l echeance courante';
20
   -- Annuaire business a venir : filtrer par genre puis par etat.
```

```

21 ALTER TABLE users
22   ADD INDEX idx_users_account_type (account_type,
23                                     verification_status);
24
25 -- Balayage du scheduler d'expiration : ne remonter que les comptes
26 -- ayant une échéance, sans parcourir toute la table.
27 ALTER TABLE users
28   ADD INDEX idx_users_verified_until (verified_until);

```

Listing 4.1 – migrations/023_account_socle.sql

Aucune colonne badge_type. Le badge n'est jamais stocké. Il se calcule à l'affichage à partir des deux axes. Une valeur stockée finirait désynchronisée le jour où une certification expire ou est révoquée — ce qui est précisément le cas que le système doit gérer.

4.3 Les énumérations, côté Node

```

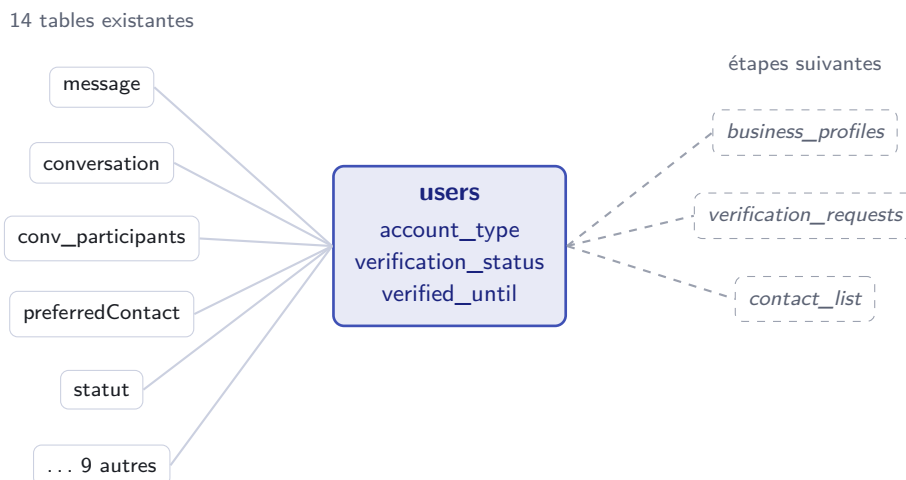
1  const ACCOUNT_TYPE = { PERSONNEL: 0, BUSINESS: 1, OFFICIEL: 2 };
2
3  const VERIFICATION = {
4    AUCUN:      0,
5    EN_ATTENTE: 1,
6    VERIFIE:    2,
7    REFUSE:     3,
8    REVOQUE:    4,
9    EXPIRE:     5,
10 };
11
12 module.exports = { ACCOUNT_TYPE, VERIFICATION };

```

Listing 4.2 – src/constants/accountTypes.js (nouveau)

Le dossier `src/constants/` existe déjà (`participantLimits.js`) : la convention est respectée.

4.4 Raccordement au schéma existant



CHAPITRE 5

Dérivation du badge

5.1 Une fonction pure, trois implémentations identiques

Le badge est une fonction de deux entiers vers une variante d’affichage. Aucun accès base, aucune requête réseau.

```
1 enum AccountBadge { aucun, cocheVerifiee, panierDeclare,
2   panierVerifie, officiel }
3
4 AccountBadge resolveAccountBadge(int accountType, int
5   verificationStatus) {
6   final verifie = verificationStatus == 2; // VERIFIE
7   switch (accountType) {
8     case 2: return AccountBadge.official;
9     case 1: return verifie ? AccountBadge.panierVerifie :
10       AccountBadge.panierDeclare;
11     default: return verifie ? AccountBadge.cocheVerifiee :
12       AccountBadge.aucun;
13   }
14 }
```

Listing 5.1 – lib/widgets/common/account_badge.dart (nouveau) — forme cible

account_type	verification_status	Variante	Rendu
0	$\neq 2$	aucun	—
0	$= 2$	cocheVerifiee	✓
1	$\neq 2$	panierDeclare	🛒
1	$= 2$	panierVerifie	⚙️
2	indifférent	officiel	🏆

Un seul widget, pas trois. Le composant doit être unique et utilisé aux quatre endroits listés en partie I. Trois implémentations concurrentes divergeront en deux semaines — c’est exactement ce qui est arrivé aux quatre indigos concurrents que le design system a dû unifier (lib/core/theme/app_colors.dart).

5.2 Le piège à corriger avant tout

Vérifié dans le code. Le champ `typeCompte` du modèle `User` est **codé en dur à 0** partout où un `User` est reconstruit depuis une charge utile partielle : `lib/core/services/call_service.dart` ligne 207, `lib/widgets/chat/contact_message_preview.dart` ligne 46, et `lib/screens/chats/contact_detail_screen.dart` ligne 104.

Les nouveaux champs suivront le même chemin et subiront le même sort : un badge dérivé afficherait « compte personnel non vérifié » à ces trois endroits, quel que soit le compte réel. Ces trois sites doivent être corrigés dans le même lot que l'ajout du badge, sinon le défaut est invisible en développement et visible en production.

5.3 Composant existant à ne pas confondre

`lib/widgets/common/app_badge.dart` existe déjà mais n'a aucun rapport : c'est `CountBadge`, la pastille de comptage des messages non lus. Le nouveau composant est donc un fichier distinct, `account_badge.dart`.

CHAPITRE 6

Règles appliquées par le serveur

6.1 Exclusion business / administrateur

Règle : `account_type != 0 ⇒ type_compte = 0`.

Pourquoi deux colonnes distinctes. `users.type_compte` existe déjà et porte le rôle d'administration (0 utilisateur, ≥ 1 admin, ≥ 2 super-admin). Il est lu par `src/middleware/adminAuth.js` (lignes 41 et 60), `lib/providers/admin_provider.dart` (lignes 245–246) et `talky-admin/lib/auth.ts`. Réutiliser cette colonne pour le genre de compte aurait cassé l'administration existante ; d'où la colonne séparée `account_type`.

Le contrôle est à poser aux deux endroits qui écrivent ces champs, dans `src/controllers/admin/users.js` : `setAccountType` (rôle admin, existant) et le nouveau point d'entrée qui fixera `account_type`. Rejet en 409 avec un message explicite plutôt qu'un écrasement silencieux.

6.2 Validation des noms d'affichage

Le filtrage doit vivre **côté serveur**, et non dans l'application mobile : le mobile n'est pas le seul écrivain. L'administration crée aussi des comptes (`src/controllers/admin/userCreate.js`), et la modification de profil passe par l'API.

```
1 // Blocs Unicode à refuser dans 'nom' et 'pseudo' :
2 //   U+2705, U+2713-U+2714 (coches)           U+2B50, U+2606, U+2B51
3 //   (etoiles)
4 //   U+1F396-U+1F397, U+1F3C5 (medailles)   U+269C (fleur de lys /
5 //   sceau)
6 //   Variation selectors utilisées pour composer un faux badge.
7 //
8 // Plus : refus des variantes normalisées du nom "Alanya", en tenant
9 //   compte
10 //   de la casse, des accents, des espaces insérées, et des homoglyphes
11 //   cyrilliques U+0410 U+0430 (a), U+0435 (e), U+043E (o), U+0443
12 //   (y),
13 //   U+0441 (c) -- visuellement identiques, distincts pour la base.
```

Listing 6.1 – `src/utils/displayNameGuard.js` (nouveau) — principe

Les homoglyphes. Refuser la chaîne « Alanya » ne suffit pas. Le même mot dont le A initial est un A cyrillique (U+0410) au lieu du A latin (U+0041) est une chaîne *différente* pour la base de données, et rigoureusement identique à l'œil. La comparaison doit donc se faire sur une forme normalisée, après repli des homoglyphes.

6.3 Interdire la réponse au compte officiel

Un seul point d’ancrage suffit. Les deux chemins d’envoi de message — REST (src/controllers/messageController.js ligne 178) et Socket.IO (src/socket/handlers/chat/messageSend.js ligne 142) — appellent tous deux la même fonction `evaluateDirectMessageSend` de `src/utils/blockUtils.js`. Une seule modification couvre donc les deux.

Cette fonction résout déjà le correspondant d’une conversation 1-1 et renvoie une action (`deliver`, `reject` ou `silent`). Il suffit d’y ajouter un cas :

```

1  const evaluateDirectMessageSend = async (conversationID, senderID) =>
2    {
3      const peerId = await getDirectConversationPeer(conversationID,
4        senderID);
5      if (peerId == null) return { isDirect: false };
6
7      // Nouveau : le compte officiel diffuse, il ne converse pas.
8      const [rows] = await pool.execute(
9        'SELECT account_type FROM users WHERE alanyaID = ?', [peerId],
10      );
11      if (rows[0] && rows[0].account_type === ACCOUNT_TYPE.OFFICIEL) {
12        return { isDirect: true, peerId, action: 'reject', code:
13          'OFFICIAL_READONLY' };
14      }
15
16      // ... suite inchangée : évaluation du blocage
17    };

```

Listing 6.2 – `src/utils/blockUtils.js` — ajout dans `evaluateDirectMessageSend`

Côté mobile, la zone de saisie de `lib/screens/chats/chat/chat_input.dart` est remplacée par un bandeau, et les actions « répondre » / « réagir » sont retirées des bulles.

À trancher à l’étape 3. Faut-il une conversation en lecture seule, ou un canal de diffusion distinct des conversations ? Le choix dépend de la stratégie de diffusion, qui est le sujet de l’étape sur le compte officiel. La règle serveur ci-dessus vaut dans les deux cas.

6.4 Expiration : réutiliser le mécanisme existant

Le comportement demandé — relance à J-30 puis suspension — a déjà son patron exact dans le code.

Ordonnanceur	src/services/meetingScheduler.js : un setInterval d'une minute, une requête qui remonte les échéances proches, et une colonne reminder_sent pour ne jamais notifier deux fois. Le squelette se transpose tel quel en verificationScheduler.js.
E-mail	src/services/mailService.js : sendMail et renderHtmlEmail sur le gabarit src/templates/email-template.html, déjà en service pour l'OTP de réinitialisation.
Notification	src/services/notificationService.js, déjà utilisé par l'ordonnanceur de réunions.
Avertissement in-app	Dérivé de verified_until, déjà présent dans la charge utile utilisateur — aucun appel supplémentaire.

```

1  -- 1. Relance J-30, une seule fois par echeance
2  SELECT alanyaID, nom, email, verified_until
3  FROM users
4  WHERE verification_status = 2
5         AND verified_until IS NOT NULL
6         AND verified_until > UTC_TIMESTAMP()
7         AND verified_until <= DATE_ADD(UTC_TIMESTAMP(), INTERVAL 30 DAY)
8         AND verification_reminder_sent = 0;
9
10 -- 2. Suspension a l'echeance
11 UPDATE users
12 SET verification_status = 5,                -- EXPIRE
13     verification_reminder_sent = 0          -- rearme pour un futur
14     renouvellement
15 WHERE verification_status = 2
16     AND verified_until IS NOT NULL
17     AND verified_until <= UTC_TIMESTAMP();

```

Listing 6.3 – Les deux requêtes du scheduler

Pourquoi pas de file de jobs ici. La conception initiale demandait une file de jobs (BullMQ / Redis) dès le départ. Elle n'existe pas aujourd'hui dans le backend. Pour les relances de vérification, le volume est faible et espacé : un ordonnanceur en processus suffit, et il a déjà fait ses preuves sur les rappels de réunion. La file redeviendra nécessaire au fan-out des diffusions du compte officiel — c'est là qu'il faudra la traiter, pas ici.

CHAPITRE 7

Impact fichier par fichier

7.1 Backend — Alanya-Backend

Fichier	Nature	Objet
migrations/023_account_socle.sql	nouveau	Les quatre colonnes et deux index
src/constants/accountTypes.js	nouveau	Énumérations partagées
src/utils/displayNameGuard.js	nouveau	Filtrage des noms, réservation d'« Alanya »
src/services/verificationScheduler.js	nouveau	Relance J-30 et suspension
src/utils/blockUtils.js	modifié	Refus de réponse au compte officiel
src/controllers/userController.js	modifié	Exposer les deux axes ; appliquer le filtrage des noms
src/controllers/conversationController.js	modifié	Ajouter les deux axes à la projection du correspondant
src/controllers/preferredContactController.js	modifié	Idem sur la liste de contacts
src/controllers/messageController.js	modifié	Idem sur l'expéditeur d'un message
src/controllers/admin/users.js	modifié	Piloter <code>account_type</code> ; exclusion business/admin ; filtre par genre
src/controllers/admin/userCreate.js	modifié	Création du compte officiel ; filtrage des noms
server.js	modifié	Démarrage du <code>verificationScheduler</code>

7.2 Mobile — Talky (Flutter)

Fichier	Nature	Objet
lib/widgets/common/account_badge.dart	nouveau	Le composant unique et sa dérivation
lib/talky_models.dart	modifié	accountType, verificationStatus, verifiedUntil sur User
lib/core/db/app_database.dart	modifié	Colonnes sur LocalUsers, schemaVersion et onUpgrade
lib/core/services/local_cache_repository.dart	modifié	Propager les champs dans le cache
lib/core/services/call_service.dart	corrigé	typeCompte: 0 codé en dur, ligne 207
lib/widgets/chat/contact_message_prev.dart	corrigé	Idem, ligne 46
lib/screens/chats/contact_detail_screen.dart	corrigé	Idem, ligne 104
lib/screens/chats/chats_screen.dart	modifié	Badge dans la liste de conversations
lib/screens/chats/chat_detail_screen.dart	modifié	Badge dans l'en-tête ; masquage de la saisie
lib/screens/chats/chat/chat_input.dart	modifié	Bandeau « diffusion » à la place du champ
lib/screens/chats/chat/chat_bubbles.dart	modifié	Retrait de « répondre » et « réagir »
lib/l10n/*	modifié	Libellés des badges, du bandeau et des avertissements

Régénération obligatoire. Toute modification de `app_database.dart` impose `dart run build_runner build --delete-conflicting-outputs`, et la montée de `schemaVersion` doit passer par `onUpgrade` — pas par une recréation, sous peine de vider le cache local des utilisateurs déjà installés.

7.3 Administration — Alanya Admin

Fichier	Nature	Objet
types/index.ts	modifié	accountType, verificationStatus, verifiedUntil
app/(dashboard)/users/page.tsx	modifié	Colonne badge et filtre par genre de compte
app/(dashboard)/users/[id]/page.tsx	modifié	Pilotage du genre et de l'état
app/(dashboard)/users/new/page.tsx	modifié	Création du compte officiel
components/ui/badge.tsx	modifié	Variante de badge de compte

Conversion des noms. Le client HTTP de l'administration convertit automatiquement les clés `snake_case` en `camelCase` (`lib/api.ts`). Les colonnes `account_type` et `verification_status` arriveront donc en `accountType` et `verificationStatus` sans travail supplémentaire.

CHAPITRE 8

Points de vigilance et questions ouvertes

8.1 À vérifier avant d'appliquer la migration

Le backend n'a pas d'ordonnanceur de migrations. Il n'existe ni exécuter, ni table de suivi : les fichiers de `migrations/` sont appliqués **à la main**. Rien ne garantit que l'état réel de la base corresponde au contenu du dossier. Avant d'appliquer 023, il faut constater l'état réel des colonnes.

La migration 013 contient du SQL invalide. `migrations/013_view_once.sql` comporte un point-virgule au milieu de l'instruction `ALTER TABLE` : la seconde colonne, `viewedAt`, n'a jamais pu s'appliquer par ce fichier. Elle figure pourtant dans le schéma consolidé `docs/schema_complet_alanyBD2027.sql`. Il faut trancher lequel des deux décrit la production.

8.2 Collision de numérotation

Le numéro 023 est également revendiqué par `docs/listes-contacts-plan.md`, qui prévoit `023_contact_lists.sql`. Le socle prenant 023, les listes de contacts passent à **024**. Le document doit être corrigé avant sa conversion, sinon les deux migrations entreraient en collision à l'implémentation.

8.3 Écart assumé avec la conception initiale

Conception initiale	Ce document
Table <code>accounts</code> dédiée « Fuir le type <code>ENUM</code> »	Extension de <code>users</code> — 14 tables et 34 contraintes en dépendent déjà Appliqué : <code>TINYINT</code> partout. À noter que le schéma existant en contient déjà (<code>migrations/020</code> et <code>021</code>) ; on ne revient pas dessus
File de jobs dès la conception	Reportée : l'ordonnanceur en processus suffit pour les relances. La file redevient nécessaire au fan-out des diffusions

8.4 Ce qui reste à décider

1. La bascule business → personnel est-elle libre, ou soumise à validation d'un administrateur ?
2. Un compte business dont la vérification a expiré reste-t-il visible dans l'annuaire business, sans badge ?
3. La durée par défaut d'une vérification — un an est l'usage, mais ce n'est pas tranché.
4. Y a-t-il un seul compte officiel, ou plusieurs (support, actualités...) ? Le modèle proposé en supporte plusieurs sans modification.

Fin du dossier — étape 1.
Étape suivante : listes de contacts préférés.